# Computational Physics _ Dr. Ejtehadi

## Homework04_RandomWalk

Mahsa Rahimi_99100671

---

### 1- Prove the Equations

$$x(t) = x(t-\tau) + a\ell$$

$$\langle x(t)\rangle = \langle x(t-\tau)\rangle + \langle a\ell\rangle \quad , \langle a\rangle = (+1)(p) + (-1)(q)$$

$$\Rightarrow \langle a\rangle = p - q$$

$$\Rightarrow \boxed{\langle x(t)\rangle = \langle x(t-\tau)\rangle + (p-q)\ell}^{*}$$

Going 1 step back: $\langle x(t-\tau)\rangle = \langle x(t-2\tau)\rangle + (p-q)\ell$

substituting in the previous equation:

$$\boxed{\langle x(t)\rangle = \langle x(t-2\tau)\rangle + 2(p-q)\ell}$$

$\Rightarrow$ In 2 steps, the RWer moves on average: $\boxed{2\times(p-q)\ell}$

and in 3 ~ ~ ~ $\boxed{3\times(p-q)\ell}$

⋮

If we start at 0 $\Rightarrow$ we need 'n' number of steps $\rightarrow n = \frac{t}{\tau}$

$$\Rightarrow t - n\tau = 0 \Rightarrow \langle x(t)\rangle = 0 + n(p-q)\ell$$

($n \to \frac{t}{\tau}$)

$$\Rightarrow \boxed{\langle x(t)\rangle = \frac{t}{\tau}(p-q)\ell}$$

5-1 Prove

$$\boxed{\sigma^2 = \langle x^2 \rangle - \langle x \rangle^2 = \frac{4l^2}{\tau} pqt}$$

Number of steps : $N = \frac{t}{\tau}$

$$a_n = \begin{cases} +1 & \text{probability } p \\ -1 & \text{probability } q \end{cases}$$

position after N steps:

$$x(t) = l \sum_{n=1}^{N} a_n$$

($x(t)$: random; $a_n$: random)

$$\langle x(t) \rangle = l \left\langle \sum_{n=1}^{N} a_n \right\rangle = l \sum_{n=1}^{N} \langle a_n \rangle$$

($\langle a_n \rangle = p - q$)

$$\rightarrow \boxed{\langle x(t) \rangle = \frac{t}{\tau}(p-q)l}$$

$$\langle x^2(t) \rangle = l^2 \left\langle \left( \sum_{n=1}^{N} a_n \right)^2 \right\rangle$$

$$\downarrow$$

$$\sum_{n=1}^{N} a_n^2 + 2 \sum a_n a_m$$

$$N + N(N-1)(p-q)^2$$

$$\Rightarrow \begin{cases} \langle x^2(t) \rangle = l^2 \left[ N + N(N-1)(p-q)^2 \right] \\ \langle x \rangle^2 = l^2 N^2 (p-q)^2 \end{cases}$$

$p + q = 1$

$$\Rightarrow \boxed{\sigma^2 = \frac{4l^2}{\tau} pqt}$$

## 2- Random Walk in 1D

General Idea: The main goal of this experiment is to implement a one-dimensional random walk and verify the correctness of the following equations for the average position and the variance of the walkers over time.

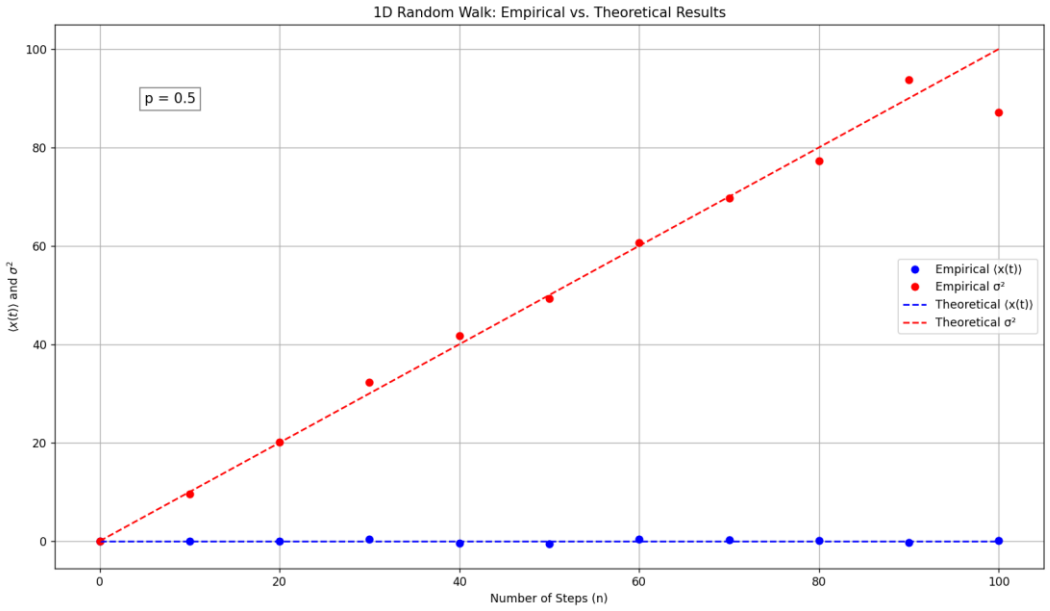
$$\langle x(t)\rangle = \langle x(0)\rangle + \frac{\tau}{l}(p-q)t$$

$$\sigma^2 = \langle x^2\rangle - \langle x\rangle^2 = \frac{\tau}{4l^2}pqt$$

The simulation is done for different values of probability p.

Algorithm and Approach: In designing the random walk simulation, we first define the key parameters: the probability $p$ of stepping to the right, the total number of steps (or time units) we want to explore, and the number of independent walkers. At each time interval, each walker decides whether to move one unit to the right or to the left based on a random number generator that compares against the probability $p$. We repeat this process for a range of step counts to build up a comprehensive set of final positions. For each walker and for each number of steps, we record the final position after all steps have been taken. Once we have these final positions, we compute the empirical mean $\langle x(t)\rangle$ and variance $\sigma^2$ across all walkers. We then compare these values to the theoretical predictions. To visualise these findings, we plot both the empirical mean and variance as functions of $t$, overlaying the corresponding theoretical curves for comparison. When the simulation is repeated for various values of $p$, we can see how different probabilities of stepping right versus left affect the expected displacement and the spread of the random walk over time.

Results and Visualisation: We can see that the empirical values match our theoretical expectations for only $p = 0.5$ .



## 2- Random Walk in 1D with Trap (Random)

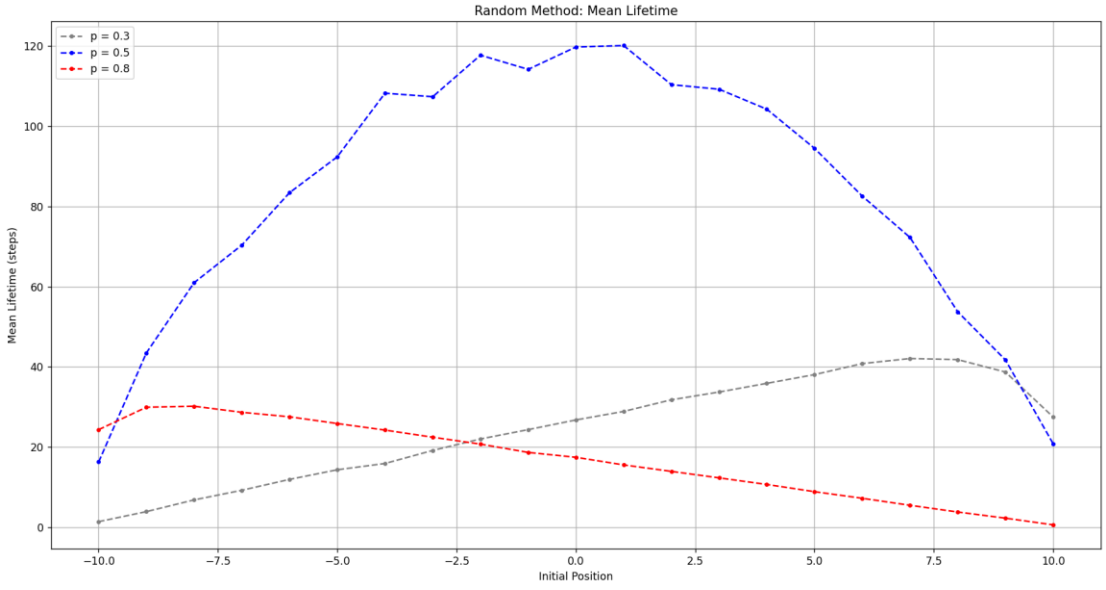
General Idea: The aim of this experiment is to analyse how the mean lifetime of a one-dimensional random walk varies with the initial position and the underlying step probability, in the presence of absorbing boundaries. In this setup, the random walker moves in a bounded interval between $-10$ and $10$. The walk terminates when the walker crosses either boundary. By adjusting the probability $p$ that governs the direction of each step, we explore different biases in the motion. The study is carried out for three different step probabilities ($p = 0.3$, $p = 0.5$, and $p = 0.8$).

Algorithm and Approach: To carry out the simulation, the approach begins by defining a function that encapsulates the decision-making process for each step. This function generates a random number uniformly between 0 and 1 and compares it with the step probability $p$. If the random number is less than or equal to $p$, the function returns a step of $+1$; otherwise, it returns $-1$.

The experiment is conducted for three different values of p ($0.3$, $0.5$, and $0.8$) to represent different degrees of bias in the walk. For each value of $p$, the simulation considers a range of initial positions from $-10$ to $10$. At each initial position, the simulation runs a large number of independent trials ($1000$ walkers) to ensure that the statistical average is robust. For each trial, the walker starts at the designated initial position and then takes successive steps determined by the aforementioned random decision function.

During each trial, the walker's journey continues until it crosses one of the predefined absorbing boundaries (either below $-10$ or above $10$). The number of steps taken before absorption is recorded as the lifetime of that particular walk. Once all trials for a given starting position are complete, the simulation calculates the mean lifetime by averaging the number of steps across all walkers.

Results and Visualisation:

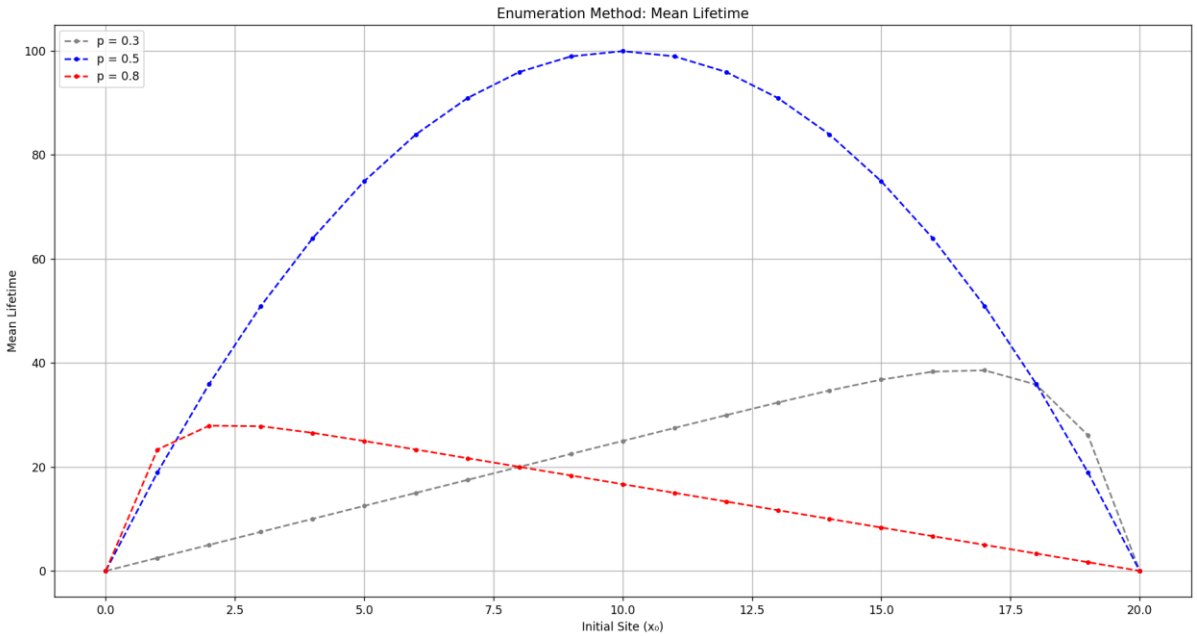


## 5- Random Walk in 1D with Trap (Enumeration)

General Idea: The goal of this experiment is to determine the mean lifetime of a random walker by evolving its probability distribution over time. This method provides a deterministic way to calculate the mean lifetime, offering an alternative approach to the stochastic simulation used in the previous exercise. In both methods, the focus is on quantifying how long, on average, a walker remains active before it is absorbed. However, while the previous method relies on averaging the outcomes of many randomly simulated trajectories, this approach deterministically updates the full probability distribution based on transition probabilities. The study is carried out for three different step probabilities ($p = 0.3$, $p = 0.5$, and $p = 0.8$).

Algorithm and Approach: We begin by initialising the probability distribution such that the walker is completely localised at a given starting site. From this initial condition, the probability distribution is updated iteratively for each time step using transition rules defined by the step probability $p$ and its complement $q = 1 - p$. At each iteration, the probability at each site is recalculated as a weighted sum of the probabilities from the adjacent sites from the previous time step. This update scheme ensures that the full evolution of the probability distribution is tracked over time. At every time step, the algorithm calculates the amount of probability that is absorbed—that is, the fraction of probability that exits the active region. By multiplying this absorbed probability by the corresponding time step and summing these contributions over all time steps, the method deterministically computes the mean lifetime for a walker starting from that position. This process is repeated for each possible initial site, and the entire procedure is carried out separately for the different values of p. The final output consists of the mean lifetime as a function of the starting site, which is then plotted for each probability.

Results and Visualisation: We get the same result as before, but it can be seen that the graph here is much smoother than the one with the previous method (random one).

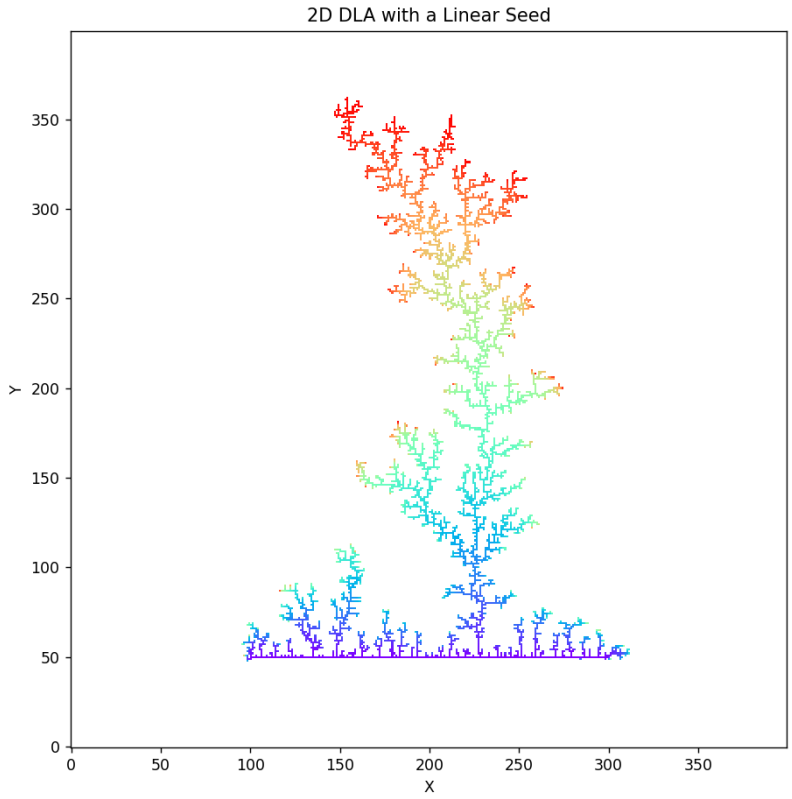


## 6- Diffusion Limited Aggregation with Linear Seed

General Idea: The goal of this experiment is to simulate the growth of a two-dimensional diffusion-limited aggregation (DLA) cluster starting from a linear seed. In this simulation, a linear seed is initially placed on a grid, and individual particles are then released to perform random walks. These particles stick to the existing cluster upon encountering any adjacent particle, leading to the formation of a fractal-like aggregate. The process is iterated over many particles to visualise the complex, branching structures characteristic of DLA.

Algorithm and Approach: We begin by defining a grid and establishing a linear seed. The seed is a contiguous line of particles placed at a specific row, and its extent is recorded to update the active area as the cluster grows. Each new particle is introduced at a random position near the top of the current cluster's boundary and then allowed to perform a random walk on the grid. At every step of the walk, the particle checks whether any of its four immediate neighbours (up, down, left, or right) belong to the existing cluster. If an adjacent particle is detected, the particle sticks to the cluster and is assigned a unique identifier, which helps track the aggregation process. This approach continues until a predetermined number of particles have been added to the cluster. The simulation also includes a bounding box check, ensuring that the random walk remains within a reasonable region relative to the growing cluster. Once all particles have aggregated, the final cluster is rendered using a rainbow colormap, providing a visually striking representation of the DLA process.

Results and Visualisation:

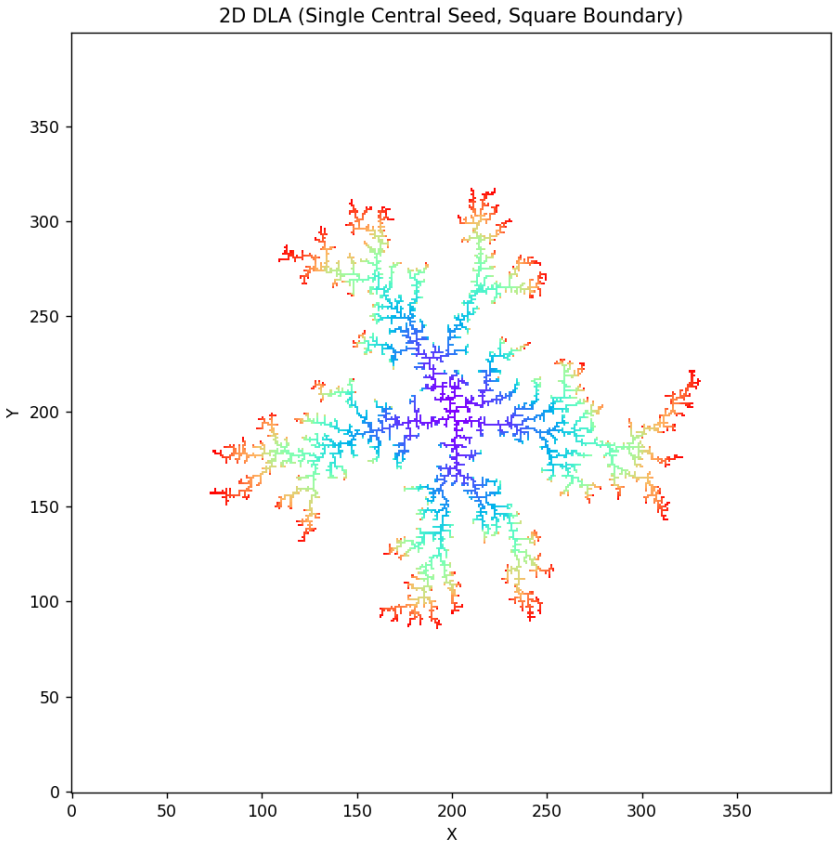


## 7- Diffusion Limited Aggregation with a Central Seed (Square Boundaries)

General Idea: This time we simulate a two-dimensional diffusion-limited aggregation (DLA) process starting from a single central seed. In this experiment, the aggregation process unfolds on a 400×400 grid where a solitary seed particle is placed at the centre. Particles are introduced from random positions along the grid's boundaries. As each particle embarks on a random walk, it continues moving until it encounters and sticks to the existing cluster.

Algorithm and Approach: The algorithm then enters a loop where, for each of the 10,000 particles, a starting position is determined by randomly selecting a coordinate along one of the grid's four boundaries. Once a particle is released, it undergoes a random walk across the grid, moving one step at a time in one of the four cardinal directions. At every step of the walk, the particle checks its immediate neighbours—above, below, to the left, and to the right—to see if any of these positions already belong to the cluster. If the particle detects an occupied neighbouring site, it adheres to the cluster at its current location. The simulation then updates the cluster's boundaries to incorporate this new addition. Additionally, a bounding box check ensures that the particle's movement remains within a predefined margin around the existing cluster, thereby preventing it from straying too far into irrelevant regions.

Results and Visualisation: It has grown mostly from sides which is a representation of square boundaries.

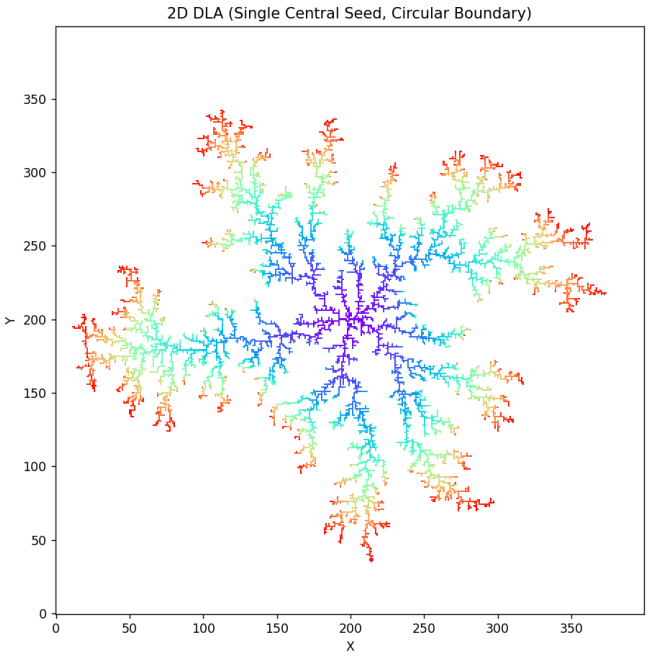


## 8- Diffusion Limited Aggregation with a Central Seed (Circle Boundaries)

General Idea: This time we simulate a two-dimensional diffusion-limited aggregation (DLA) process using a circular boundary for particle injection. In this experiment, the grid is again a $400 \times 400$ array with a single central seed placed at the centre. Unlike the previous cases where particles were injected from a square boundary, here the particles are introduced from a circular boundary defined by a fixed radius around the centre. Each particle performs a random walk until it adheres to the existing cluster upon encountering a neighbouring occupied site. This setup allows us to explore how a circular injection boundary influences the growth and structure of the resulting DLA cluster, leading to a distinct pattern that reflects the isotropy of the circular boundary.

Algorithm and Approach: In this simulation, the algorithm begins by initialising the grid and positioning the seed particle at the centre of the $400 \times 400$ grid. The injection method is modified to select starting positions randomly along a circular boundary, which is achieved by generating random angles and computing the corresponding $x$ and $y$ coordinates based on a fixed radius from the centre. This method ensures that particles are introduced from all directions in a uniform manner. Once a particle is released from the circular boundary, it embarks on a random walk across the grid. At each step, the particle evaluates its four immediate neighbours (up, down, left, right) to determine if it has reached the existing cluster. If a neighbouring cell is occupied, the particle sticks to the cluster and is assigned a unique identifier to record its order of adhesion. The boundaries of the cluster are updated dynamically as new particles attach, ensuring that the simulation tracks the growing extent of the aggregate. A bounding box check with a margin is also applied to prevent particles from straying too far from the cluster.

Results and Visualisation: We can see that it has grown from every direction, due to the fact that it had a circle as boundary.



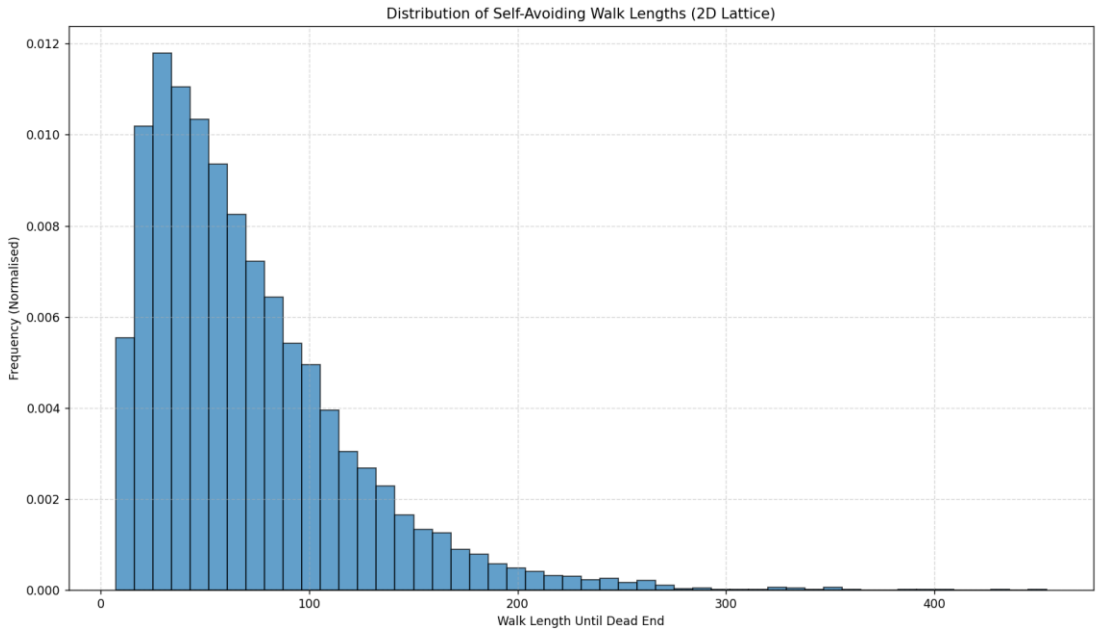
## 9- Self-Avoiding Random Walk

General Idea: This experiment simulates self-avoiding walks (SAWs) on a two-dimensional lattice. The main objective is to investigate the distribution of walk lengths achieved before the walker reaches a dead end—that is, a point where no further moves are available without revisiting a previously occupied site. In each trial, a walker starts at the origin and takes successive steps in one of the four cardinal directions (up, down, left, or right), ensuring that no position is visited more than once.

Algorithm and Approach: Each walk begins at the origin, with the starting position recorded in a set to track visited sites. At every step, the algorithm evaluates the four possible moves corresponding to the cardinal directions and filters out any moves that would lead to a previously visited site. This creates a list of valid moves that maintain the self-avoiding condition. If there are no valid moves available, the walk is terminated, and the number of steps taken is recorded as the walk length. This process is repeated for 10,000 independent trials, resulting in a collection of walk lengths. A histogram of these lengths is then plotted to visualise the frequency distribution, and summary statistics such as the mean and maximum walk length are printed. By analysing this distribution, we gain an understanding of the typical extent of self-avoiding walks and how often walkers get trapped early versus reaching longer paths before encountering a dead end.

Results and Visualisation: As the walk progresses, it increasingly restricts its own movement because each visited site cannot be revisited. This causes the number of available steps to shrink over time, eventually leading to a dead end. Consequently, the histogram shows that most walks terminate relatively quickly, evidenced by the peak at lower walk lengths. (Although a smaller number of walks manage to continue longer.)

Moreover, for a 10,000 number of walks, I got 71.03 for the mean walk length and 448 for the max walk length from the data.



## 10- SAW vs N

General Idea: In this experiment, we enumerate self-avoiding walks (SAWs) on a two-dimensional lattice to understand how the number of such walks grows with the walk length $N$. The primary objective is to compute the total number of self-avoiding paths of length $N$ starting from the origin and to compare this count to the total number of unrestricted random walks (given by $4^N$, since there are four possible moves at each step).

Algorithm and Approach: Here a recursive depth-first search (DFS) strategy is used to count all self-avoiding walks of a given length $N$ on a $2D$ lattice. Starting at the origin, the algorithm explores each possible move in the four cardinal directions (up, down, left, and right). To maintain the self-avoiding condition, the algorithm tracks all visited sites in a set and only considers moves that lead to previously unvisited coordinates. When the walk reaches the target length $N$, it increments the count by one, representing a complete self-avoiding path. After obtaining the count of SAWs for each walk length from 1 to 15, the algorithm calculates the ratio of SAWs to the total number of free random walks ($4^N$) for each $N$.